
SEGMENT 1

Structure of Compiler

1. Programming Languages and Translation

Computers **do not understand high-level languages** like C, Java, or Python.

Computers understand **machine language**:

```
101101010  
110010011
```

But humans write programs in **high-level languages**:

```
int a = b + c;
```

Therefore we need **translation software**.

This translation software is called a **translator**.

2. Translator

A **translator** is a system program that converts a program written in one language into another language.

Definition:

A translator is a software that converts a source program written in one programming language into an equivalent program in another language.

Usually:

```
High Level Language → Machine Language
```

Types of Translators

There are **three major translators**.

1. Compiler

2. Interpreter

3. Assembler

3. Compiler

A **compiler** translates the **entire program at once** from a high-level language into machine code.

Definition

A compiler is a system software that translates a high-level language program into machine code before execution.

Example languages compiled:

- C
- C++
- Go
- Rust

Example process:

```
C program
  ↓
Compiler
  ↓
Machine code
```

Example

Source code:

```
int a = 5;
int b = 10;
int c = a + b;
```

Compiler converts it to:

```
LOAD 5
STORE A
LOAD 10
STORE B
ADD A,B
STORE C
```

Characteristics of Compiler

1. Entire program compiled at once
 2. Generates object code
 3. Errors reported after compilation
 4. Execution is faster
-

4. Interpreter

An **interpreter** translates and executes a program **line by line**.

Definition:

An interpreter is a translator that converts high-level instructions into machine instructions one statement at a time.

Example languages:

- Python
 - JavaScript
 - MATLAB
-

Example

Program:

```
a = 5
b = 10
print(a+b)
```

Interpreter:

```
execute line 1
execute line 2
execute line 3
```

Characteristics

1. Line-by-line translation
 2. No executable file generated
 3. Slower execution
 4. Errors reported immediately
-

5. Assembler

An **assembler** converts assembly language into machine code.

Example:

Assembly code:

```
MOV A, 5
ADD A, B
```

Assembler converts this to machine code.

6. Compiler vs Interpreter

Feature	Compiler	Interpreter
Translation	Whole program	Line by line
Speed	Faster	Slower
Output	Executable	No executable
Error detection	After compilation	Immediate

7. Basic Principles of Compiler Design

Compiler design follows several important principles.

These principles ensure that the compiler is:

- efficient
- modular
- maintainable
- portable

The important ideas are:

1 Modular Design

A compiler is divided into **separate phases**.

Example:

```
Lexical analysis
Syntax analysis
Semantic analysis
```

Each phase performs a **specific task**.

2 Machine Independence

The compiler should separate:

```
language analysis
machine code generation
```

So the same compiler can target different machines.

3 Error Detection and Recovery

A compiler must detect:

- syntax errors
- semantic errors

Example:

```
int a
a = 5
```

Missing semicolon.

4 Optimization

The compiler should improve program performance.

Example:

```
x = y * 1
```

Optimized to

```
x = y
```

8. Applications of Compilers

Compilers are used in many areas.

1 Programming Languages

Compilers translate languages such as:

- C
 - C++
 - Java
 - Rust
-

2 Embedded Systems

Embedded devices require compiled programs.

Example:

- microcontrollers
 - IoT devices
-

3 Operating Systems

Operating systems are written using compiled languages.

Example:

- Linux kernel written in C
-

4 Database Systems

Database query languages use compiler techniques.

Example:

SQL queries are parsed and optimized.

5 Artificial Intelligence

Machine learning frameworks use compiler techniques.

Example:

TensorFlow compiler.

9. Language Processing System

A **language processing system** converts a source program into an executable program.

The process is not done only by the compiler.

There are **multiple tools involved**.

Structure

```
Source Program
  ↓
Compiler
  ↓
Assembly Code
  ↓
Assembler
  ↓
Object Code
  ↓
Linker
  ↓
Executable Program
```

10. Components of Language Processing System

1 Compiler

Converts high-level language to assembly code.

2 Assembler

Converts assembly language to machine code.

3 Linker

Combines object files with libraries.

Example:

```
printf()
math functions
```

4 Loader

Loads executable program into memory.

11. Phases of Compiler

A compiler performs translation in **multiple phases**.

These phases are designed to simplify the compilation process.

Main phases:

- 1 Lexical Analysis
 - 2 Syntax Analysis
 - 3 Semantic Analysis
 - 4 Intermediate Code Generation
 - 5 Code Optimization
 - 6 Code Generation
-

12. Structure of Compiler

```
Source Program
  ↓
Lexical Analysis
  ↓
Syntax Analysis
  ↓
Semantic Analysis
  ↓
Intermediate Code
  ↓
Code Optimization
  ↓
Target Code Generation
```

13. Front End and Back End of Compiler

The compiler is divided into **two main parts**.

Front End

The front end analyzes the program.

It is **machine independent**.

Front end phases:

Lexical Analysis
Syntax Analysis
Semantic Analysis

Responsibilities:

- recognize tokens
- check grammar
- check meaning

Back End

The back end generates machine code.

It is **machine dependent**.

Back end phases:

Intermediate Code Generation
Code Optimization
Code Generation

Responsibilities:

- generate intermediate code
- optimize program
- generate machine instructions

14. Example Translation Through Compiler Phases

Example statement from your past questions:

`a = x + y * r - w / 4`

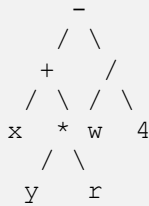
Step 1 Lexical Analysis

Tokens identified:

```
id(a)
=
id(x)
+
id(y)
*
id(r)
-
id(w)
/
num(4)
```

Step 2 Syntax Analysis

Parser constructs expression tree.



Step 3 Semantic Analysis

Checks:

- variable types
- operator compatibility

Example:

```
x,y,r,w are float
```

Step 4 Intermediate Code

Three-address code:

```
t1 = y * r
t2 = x + t1
t3 = w / 4
t4 = t2 - t3
a = t4
```

Step 5 Optimization

Example:

```
remove unnecessary temporary variables
```

Step 6 Code Generation

Machine instructions generated.

15. Symbol Table

A symbol table stores information about identifiers.

Example table:

Identifier	Type	Scope	Address
a	float	local	1000
x	float	local	1004

Symbol table is used by:

- lexical analyzer
 - parser
 - semantic analyzer
-

SEGMENT 2

Grammars & Expressions

1. Formal Languages

A **language** in compiler theory is a **set of strings formed from an alphabet**.

A string is a sequence of symbols.

Example alphabet:

$$\Sigma = \{a, b\}$$

Example strings:

a
b
ab
aab
abb

Example language:

$$L = \{a^n b^n \mid n \geq 1\}$$

Strings in the language:

ab
aabb
aaabbb
aaaabbbb

2. Alphabet, String and Language

These three terms are fundamental.

Alphabet (Σ)

A finite set of symbols.

Example

$$\Sigma = \{0, 1\}$$

or

$$\Sigma = \{a, b, c\}$$

String

A sequence of symbols from the alphabet.

Example

0101
abba

The empty string is written as

ε

Language

A set of strings over an alphabet.

Example

$$L = \{0, 1, 00, 11, 010\}$$

3. Grammar

A **grammar** is a formal system used to define a language.

Definition:

A grammar is a set of rules used to generate all valid strings of a language.

A grammar is defined as:

$$G = (V, T, P, S)$$

Where

V = non-terminals

T = terminals

P = production rules

S = start symbol

4. Terminals and Non-Terminals

Terminals

Terminals are the **actual symbols that appear in the final strings**.

Example

a b + * id

These cannot be replaced further.

Non-Terminals

Non-terminals are **variables used in grammar rules**.

Example

S
A
B

E
T
F

They help generate the structure of the language.

Example Grammar

$S \rightarrow aSb$
 $S \rightarrow ab$

Here

Terminals

a b

Non-terminals

S

5. Production Rules

Production rules describe **how symbols can be replaced**.

General form

$A \rightarrow \alpha$

Where

A = non-terminal

α = terminals and/or non-terminals

Example

$S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow b$

6. Derivation

Derivation is the process of generating strings from the start symbol.

Example grammar

$S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow b$

Derivation

$S \rightarrow AB$
 $\rightarrow aB$
 $\rightarrow ab$

7. Leftmost Derivation

In leftmost derivation, **the leftmost non-terminal is expanded first.**

Example grammar

$S \rightarrow AB$
 $A \rightarrow aA \mid a$
 $B \rightarrow b$

Derivation

$S \rightarrow AB$
 $\rightarrow aAB$
 $\rightarrow aaB$
 $\rightarrow aab$

At each step the **leftmost variable is replaced.**

8. Rightmost Derivation

In rightmost derivation, **the rightmost non-terminal is expanded first.**

Example

$S \rightarrow AB$
 $A \rightarrow a$

$B \rightarrow bB \mid b$

Rightmost derivation

$S \rightarrow AB$
 $\rightarrow AbB$
 $\rightarrow Abb$
 $\rightarrow abb$

9. Parse Tree

A **parse tree** represents the syntactic structure of a string.

Example grammar

$E \rightarrow E + T$
 $E \rightarrow T$
 $T \rightarrow id$

String

`id + id`

Parse tree

```
      E
     /\
    E + T
    |   |
    T   id
    |
    id
```

Properties of parse tree

1. Root is start symbol
 2. Internal nodes are non-terminals
 3. Leaves are terminals
-

10. Context-Free Grammar (CFG)

A **context-free grammar** has rules of the form

$$A \rightarrow \alpha$$

Where

A = single non-terminal

α = string of terminals and non-terminals

CFG is used to describe **programming language syntax**.

Example CFG for Arithmetic Expressions

$$E \rightarrow E + T$$
$$E \rightarrow T$$
$$T \rightarrow T * F$$
$$T \rightarrow F$$
$$F \rightarrow (E)$$
$$F \rightarrow \text{id}$$

This grammar defines arithmetic expressions.

Example valid strings

`id + id`

`id * id`

`(id + id) * id`

11. Ambiguous Grammar

A grammar is **ambiguous** if a string has more than one parse tree.

Example grammar

$$E \rightarrow E + E$$
$$E \rightarrow E * E$$
$$E \rightarrow \text{id}$$

Input string

`id + id * id`

Two interpretations

$(id + id) * id$

or

$id + (id * id)$

Therefore grammar is ambiguous.

12. Removing Ambiguity

Ambiguity can be removed by **defining operator precedence**.

Example unambiguous grammar

```
E → E + T | T
T → T * F | F
F → (E) | id
```

Here

* has higher precedence than +

13. Context-Sensitive Grammar

Context-sensitive grammars are more powerful than CFG.

Rule form

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

Where

$$\gamma \neq \varepsilon$$

Meaning the replacement of **A** **depends on surrounding symbols**.

Example

$$AB \rightarrow BA$$

14. Regular Grammar

Regular grammars are the simplest type.

Rules have the form

$$\begin{aligned} A &\rightarrow aB \\ A &\rightarrow a \end{aligned}$$

or

$$\begin{aligned} A &\rightarrow Ba \\ A &\rightarrow a \end{aligned}$$

Regular grammars describe **regular languages**.

These correspond to **finite automata**.

15. Right Linear Grammar

Right linear grammar has rules of form

$$\begin{aligned} A &\rightarrow aB \\ A &\rightarrow a \end{aligned}$$

Example

$$\begin{aligned} S &\rightarrow aA \\ A &\rightarrow bB \\ B &\rightarrow c \end{aligned}$$

The non-terminal appears on the **right side**.

16. Left Linear Grammar

Left linear grammar has rules of form

$$\begin{aligned} A &\rightarrow Ba \\ A &\rightarrow a \end{aligned}$$

Example

$S \rightarrow Ab$
 $A \rightarrow Ba$
 $B \rightarrow a$

The non-terminal appears on the **left side**.

17. Chomsky Hierarchy

Grammars are classified into four types.

Type	Grammar	Language
Type 0	Unrestricted	Recursively enumerable
Type 1	Context sensitive	CSL
Type 2	Context free	CFL
Type 3	Regular	RL

18. Useless Non-Terminals

A non-terminal is useless if it **cannot contribute to generating terminal strings**.

Two types exist.

1 Non-Generating Symbols

Symbols that **cannot produce terminal strings**.

Example

$S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow C$
 $C \rightarrow C$

Here

C cannot produce terminals

So

B and C are useless

2 Inaccessible Symbols

Symbols that **cannot be reached from the start symbol**.

Example

$S \rightarrow A$
 $A \rightarrow a$
 $B \rightarrow b$

Here

`B is inaccessible`

because it never appears in derivations from S .

19. Removing Useless Symbols

Two steps:

- 1 Remove non-generating symbols
 - 2 Remove unreachable symbols
-

20. Example From Your Past Questions

Example grammar

$S \rightarrow aA \mid bB$
 $A \rightarrow aA \mid \epsilon$
 $B \rightarrow bB \mid \epsilon$

This grammar generates

`an or bn`

Examples

`a`
`aa`
`aaa`

b
bb
bbb

21. CFG Example: Palindrome Language

Your previous question asked:

Define CFG for palindrome strings of 0 and 1

A palindrome reads same forward and backward.

Examples

0
1
00
11
101
0110

CFG

$S \rightarrow 0S0$
 $S \rightarrow 1S1$
 $S \rightarrow 0$
 $S \rightarrow 1$
 $S \rightarrow \epsilon$

22. Example Problem From Your Exam

Grammar

$\text{Exp} \rightarrow \text{Exp} + \text{Exp}$
 $\text{Exp} \rightarrow \text{Exp} * \text{Exp}$
 $\text{Exp} \rightarrow \text{id}$

Input

`id + id * id`

Two parse trees possible.

Therefore

grammar is ambiguous

To fix

```
E → E + T
T → T * F
F → id
```

23. Robot Movement Grammar (From Your Question)

Grammar

```
Seq → Seq Instr | begin
Instr → east | north | west | south
```

This grammar generates movement instructions.

Example string

```
begin west south east east north north
```

Each instruction moves robot position.

Example coordinates

```
start (0,0)

west  → (-1,0)
south → (-1,-1)
east  → (0,-1)
east  → (1,-1)
north → (1,0)
north → (1,1)
```

Final position

```
(1,1)
```

SEGMENT 3

Automata & Lexical Analysis

1. Lexical Analysis

Lexical analysis is the **first phase of the compiler**.

It reads the **source program character by character** and groups them into **tokens**.

Example code:

```
int c, n = 5;
```

Lexical analyzer converts this into tokens:

```
keyword(int)
identifier(c)
separator(,)
identifier(n)
operator(=)
number(5)
separator(;
```

Thus the lexical analyzer converts **characters** → **tokens**.

2. Role of Lexical Analyzer

The lexical analyzer performs several tasks.

1. Tokenization

Breaking the input stream into tokens.

Example

```
a = b + 10
```

Tokens:

```
id  
=  
id  
+  
num
```

2. Removing whitespace and comments

Example

```
int a;          // comment
```

Whitespace and comments are ignored.

3. Error detection

Example:

```
int 9abc;
```

Invalid identifier.

4. Symbol table interaction

Identifiers are stored in the **symbol table**.

Example entry:

Name	Type	Address
a	int	1000

5. Interface with parser

Lexical analyzer passes tokens to the parser.

3. Tokens, Lexemes and Patterns

These are fundamental concepts in lexical analysis.

Token

A **token** is a category of lexemes.

Example token types:

```
identifier  
keyword  
operator  
number  
separator
```

Lexeme

A lexeme is the **actual sequence of characters**.

Example

```
int  
x  
=  
10  
+
```

Pattern

A **pattern** describes the rule for lexemes.

Example pattern for identifier

```
letter(letter|digit)*
```

Example Table (from your exam questions)

Code:

```
int c, n=5;
```

Token	Lexeme	Pattern
keyword	int	reserved word
identifier	c	letter(letter
separator	,	,
identifier	n	letter(letter
operator	=	=
number	5	digit+

4. Regular Expressions

Regular expressions describe **patterns of strings**.

They are used to define tokens.

Basic operators:

Operator	Meaning
,	,
*	zero or more
+	one or more
()	grouping

Examples

Identifier

`[a-zA-Z_][a-zA-Z0-9_]*`

Meaning

- first character letter or _
 - rest letters or digits
-

Integer

`[0-9]+`

Floating number

$[0-9]^+\backslash.[0-9]^+$

Binary numbers

$(0|1)^+$

Example from your exam

Regular expression:

$(a|b)^*abb$

Meaning:

Any number of **a or b**, ending with **abb**

Examples:

abb
aabb
bababb

5. Finite Automata

Finite automata are mathematical machines used to recognize languages.

They are widely used in **lexical analysis**.

A finite automaton is defined as:

$M = (Q, \Sigma, \delta, q_0, F)$

Where

Q = set of states
 Σ = input alphabet
 δ = transition function
 q_0 = start state
 F = set of final states

6. Deterministic Finite Automaton (DFA)

A DFA is a finite automaton where **each state has exactly one transition for each input symbol**.

Properties:

- No ϵ transitions
- Deterministic behavior

Example DFA:

```
q0 --a--> q1
q1 --b--> q2
```

String accepted: **ab**

7. Non-Deterministic Finite Automaton (NFA)

An NFA allows:

- multiple transitions
- ϵ transitions

Example:

```
q0 --a--> q1
q0 --a--> q2
```

For input **a**, two possible states exist.

8. Difference Between DFA and NFA

Feature	DFA	NFA
transitions	one per symbol	many allowed
ϵ transitions	not allowed	allowed
complexity	larger	smaller
implementation	easier	theoretical

Both recognize the **same class of languages (regular languages)**.

9. ϵ -NFA

An ϵ -NFA is an NFA that allows **epsilon transitions**.

Epsilon means **state change without consuming input**.

Example:

```
q0 --ε--> q1
q1 --a--> q2
```

The machine can move from **q0 to q1 without reading input**.

10. ϵ -Closure

The ϵ -closure of a state is the set of states reachable through **only ϵ transitions**.

Example

```
q0 --ε--> q1
q1 --ε--> q2
```

Then

```
ε-closure(q0) = {q0, q1, q2}
```

11. Converting ϵ -NFA to DFA

The conversion is done using **subset construction**.

Steps:

1. Compute ϵ -closure of start state
2. Use this as first DFA state
3. Compute transitions for each input
4. Compute ϵ -closure of resulting states
5. Continue until no new states appear

Final states contain any **NFA final state**.

12. Regular Expression $\rightarrow$ NFA

Regular expressions can be converted to NFA using **Thompson's construction**.

Example regex:

`(a | b) * abb`

Structure:

1. NFA for `a | b`
2. apply Kleene star
3. concatenate `abb`

This forms a machine that accepts strings ending with **abb**.

13. Automaton for C Identifier

Regular expression:

`[a-zA-Z_] [a-zA-Z0-9_]*`

Meaning:

1. First character letter or underscore

2. Remaining characters letters or digits

State logic

```
start → letter/_ → identifier state  
identifier → letter/digit/_ → stay
```

14. Automaton for Numbers

Integer regular expression

`digit+`

Example DFA

```
q0 --digit--> q1  
q1 --digit--> q1
```

Final state: q1

15. Finite Automata Minimization

Minimization reduces the number of states of a DFA.

Goal:

create smallest equivalent DFA

Steps

1. Separate final and non-final states
2. Split states based on transitions
3. Merge equivalent states

Result:

Minimal DFA.

16. Importance of Finite Automata in Compiler

Finite automata are important because they help:

1. Recognize tokens
2. Implement lexical analyzers
3. Perform pattern matching
4. Build scanners automatically

Lexical analyzer generators such as:

LEX
FLEX

use finite automata.

17. Symbol Table Management

The symbol table stores information about program identifiers.

Example

Name	Type	Scope	Address
x	int	local	100
y	float	global	200

Operations on Symbol Table

Insert

Add new identifier.

Lookup

Check if identifier already exists.

Update

Modify attributes.

Delete

Remove entries when scope ends.

18. Interaction of Lexical Analyzer with Symbol Table

Example code:

```
int c;
```

Process:

1. lexical analyzer identifies **identifier c**
2. symbol table entry created

Example entry

```
name: c  
type: int  
address: memory location
```

.